Blockchain Lab
EXP 7

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

Blockchain is a decentralized and distributed digital ledger that securely records transactions across multiple systems. It ensures transparency, security, and immutability, meaning that once data is recorded, it cannot be altered.

Ethereum is a widely used blockchain platform that enables the execution of smart contracts. Smart contracts are self-executing programs written in Solidity that automatically carry out actions when predefined conditions are met, eliminating the need for intermediaries.

MetaMask is a cryptocurrency wallet and browser extension used to interact with blockchain networks. It securely stores user accounts and private keys, and allows users to send, receive, and sign transactions. It also serves as a bridge between Remix IDE and blockchain networks.

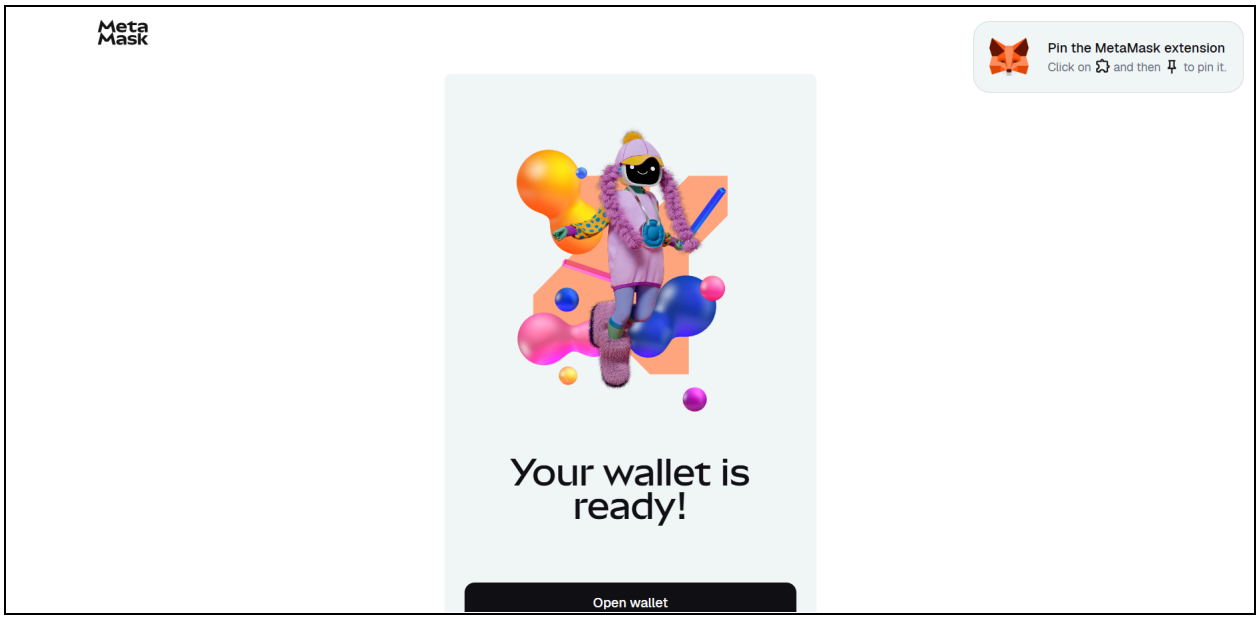
Test networks such as Sepolia and RSK Testnet are used for development and testing. They provide free tokens, enabling developers to deploy and test smart contracts without using real cryptocurrency, thereby reducing financial risk.

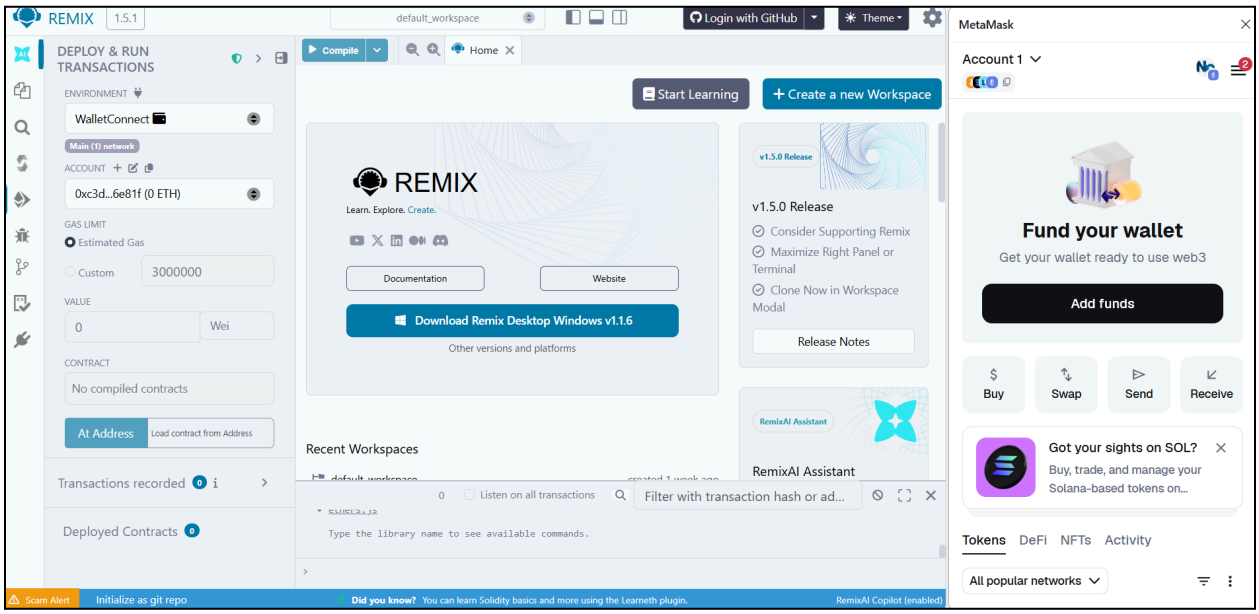
Remix IDE is a web-based development environment used to write, compile, deploy, and test smart contracts. It offers a user-friendly interface and integrates directly with MetaMask for deploying contracts on blockchain networks.

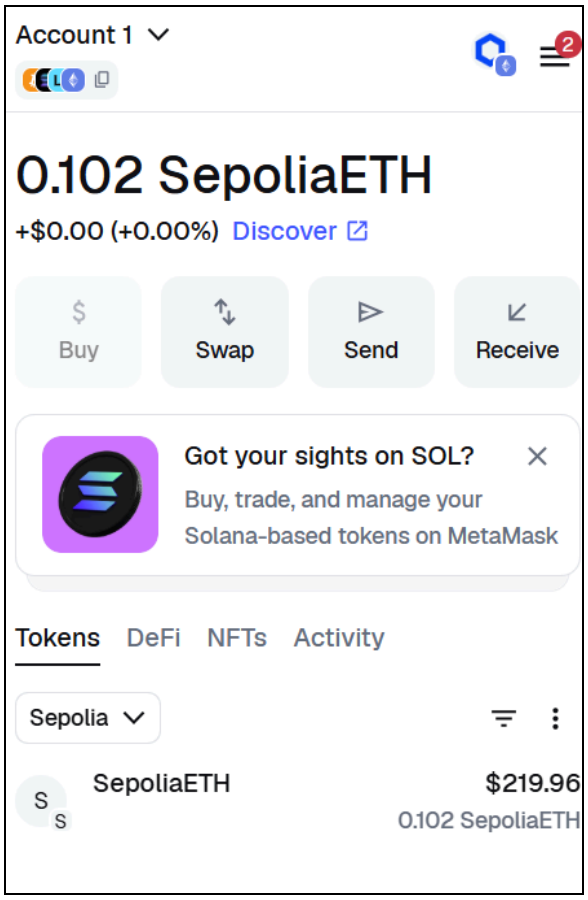
The smart contract deployment process involves writing the contract in Solidity, compiling it to eliminate errors, connecting MetaMask to Remix, and deploying the contract on a testnet by paying gas fees with test tokens.

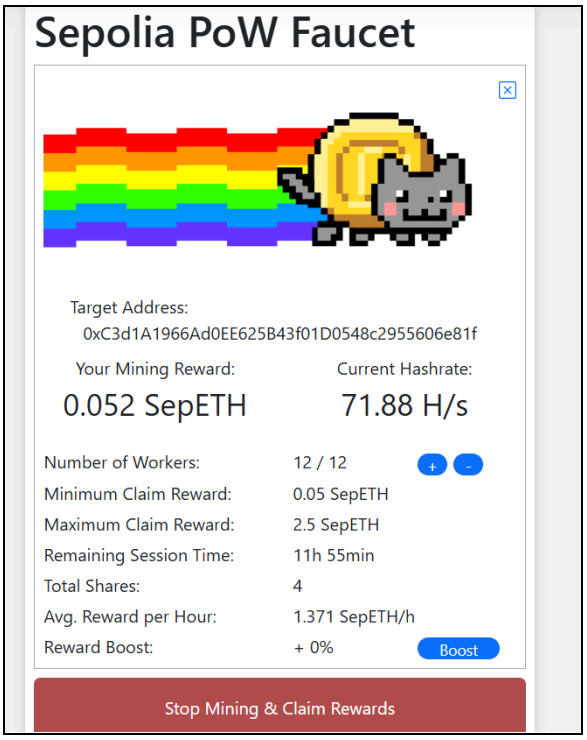
A blockchain transaction refers to any interaction with the network, such as deploying a contract or executing its functions. Each transaction requires authentication through a digital signature and is permanently recorded on the blockchain. These transactions can be verified using blockchain explorers.

Step 1 —Install MetaMask
Install Metamask and create wallet


Meta Mask
Pin the MetaMask extension
Click on and then to pin it.
Your wallet is ready!
Open wallet

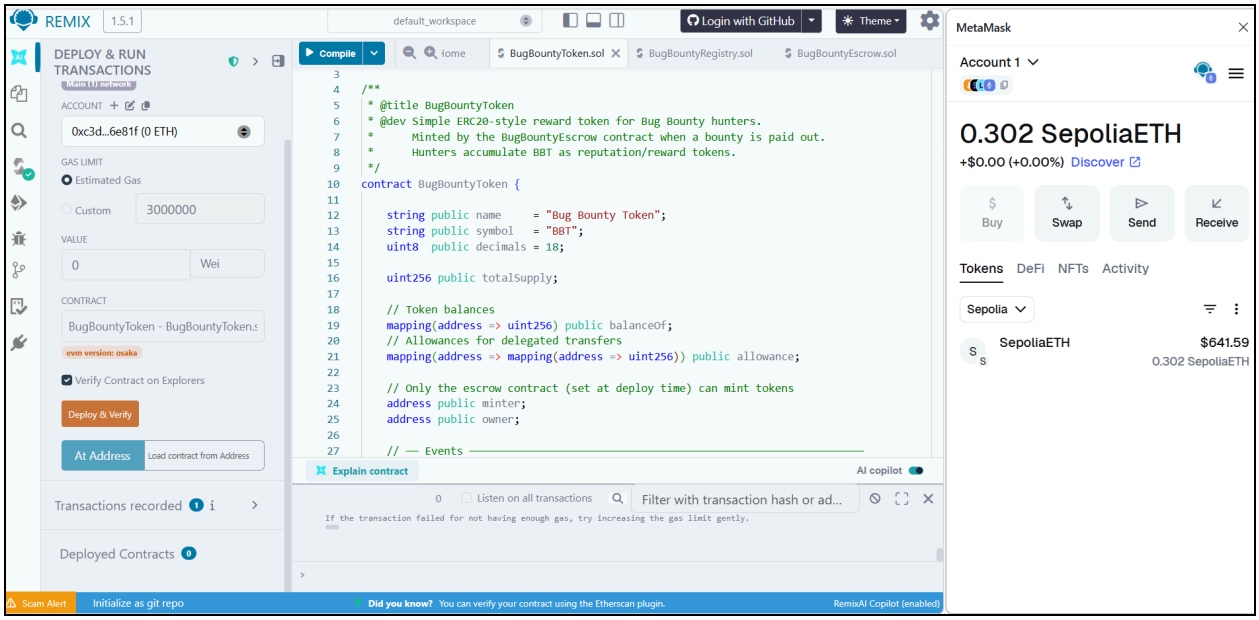

REMIX 1.5.1
default_workspace
Login with GitHub
Theme
DEPLOY & RUN TRANSACTIONS
ENVIRONMENT
WalletConnect
Main (1) network
ACCOUNT
0xc3d...6e81f (0 ETH)
GAS LIMIT
Estimated Gas
Custom
3000000
VALUE
0
Wei
CONTRACT
No compiled contracts
At Address
Load contract from Address
Transactions recorded
Deployed Contracts
Compile
Home
Start Learning
Create a new Workspace
REMIX
Learn. Explore. Create.
Documentation
Website
Download Remix Desktop Windows v1.1.6
Other versions and platforms
v1.5.0 Release
Consider Supporting Remix
Maximize Right Panel or Terminal
Clone Now in Workspace Modal
Release Notes
RemixAI Assistant
Recent Workspaces
Listen on all transactions
Filter with transaction hash or ad...
Type the library name to see available commands.
Scam Alert
Initialize as git repo
Did you know? You can learn Solidity basics and more using the Learneth plugin.
RemixAI Copilot (enabled)
MetaMask
Account 1
Fund your wallet
Get your wallet ready to use web3
Add funds
Buy
Swap
Send
Receive
Got your sights on SOL?
Buy, trade, and manage your Solana-based tokens on...
Tokens
DeFi
NFTs
Activity
All popular networks

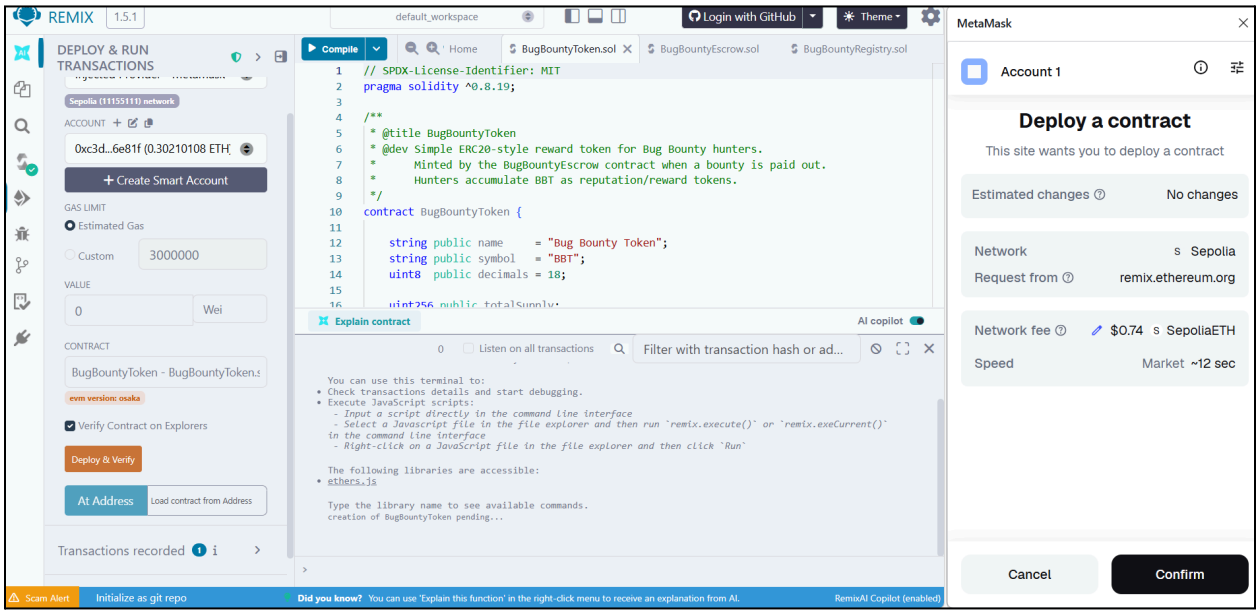


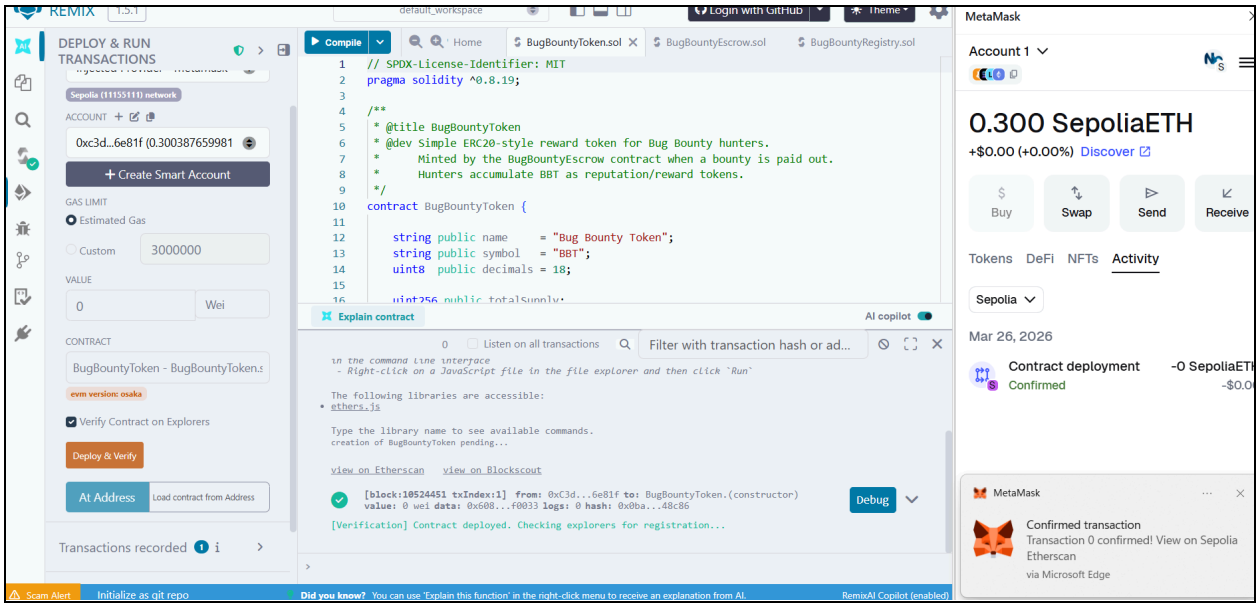


Step 2 — Deploy BugBountyToken First

1. In Remix, go to the **Deploy tab** (Ethereum icon on left)
2. Make sure Environment is **"Injected Provider - MetaMask"**
3. In the **Contract dropdown**, select **"BugBountyToken"**
4. Click **Deploy**
5. MetaMask will pop up → click **Confirm**
6. Wait a few seconds → you'll see it appear under **"Deployed Contracts"**
7. **Copy and save its address** (click the copy icon next to it)

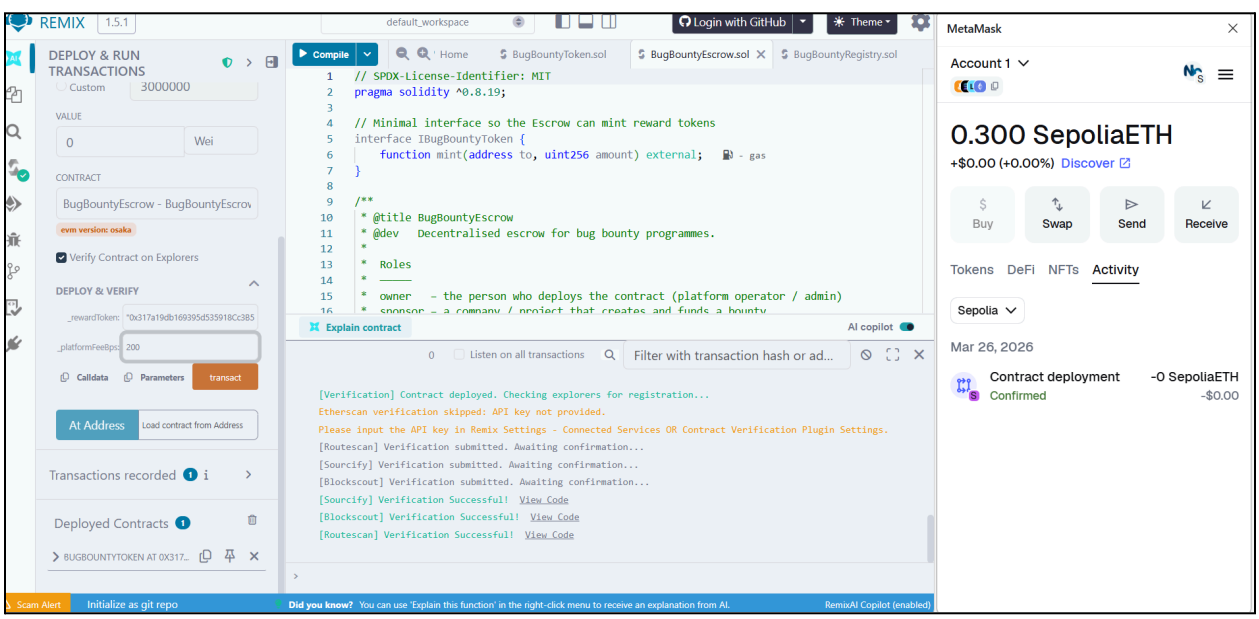

REMIX 1.5.1
default_workspace
Login with GitHub
Theme
DEPLOY & RUN TRANSACTIONS
Main (1) network
ACCOUNT
0xc3d...6e81f (0 ETH)
GAS LIMIT
Estimated Gas
Custom
3000000
VALUE
0
Wei
CONTRACT
BugBountyToken - BugBountyToken.s
evm version: osaka
Verify Contract on Explorers
Deploy & Verify
At Address
Load contract from Address
Transactions recorded 1
Deployed Contracts 0
Compile
Home
BugBountyToken.sol
BugBountyRegistry.sol
BugBountyEscrow.sol
/**
* @title BugBountyToken
* @dev Simple ERC20-style reward token for Bug Bounty hunters.
* Minted by the BugBountyEscrow contract when a bounty is paid out.
* Hunters accumulate BBT as reputation/reward tokens.
*/
contract BugBountyToken {
string public name = "Bug Bounty Token";
string public symbol = "BBT";
uint8 public decimals = 18;
uint256 public totalSupply;
// Token balances
mapping(address => uint256) public balanceOf;
// Allowances for delegated transfers
mapping(address => mapping(address => uint256)) public allowance;
// Only the escrow contract (set at deploy time) can mint tokens
address public minter;
address public owner;
// — Events
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
If the transaction failed for not having enough gas, try increasing the gas limit gently.
Scam Alert
Initialize as git repo
Did you know? You can verify your contract using the Etherscan plugin.
RemixAI Copilot (enabled)
MetaMask
Account 1
0.302 SepoliaETH
+$0.00 (+0.00%) Discover
Buy
Swap
Send
Receive
Tokens
DeFi
NFTs
Activity
Sepolia
SepoliaETH
$641.59
0.302 SepoliaETH

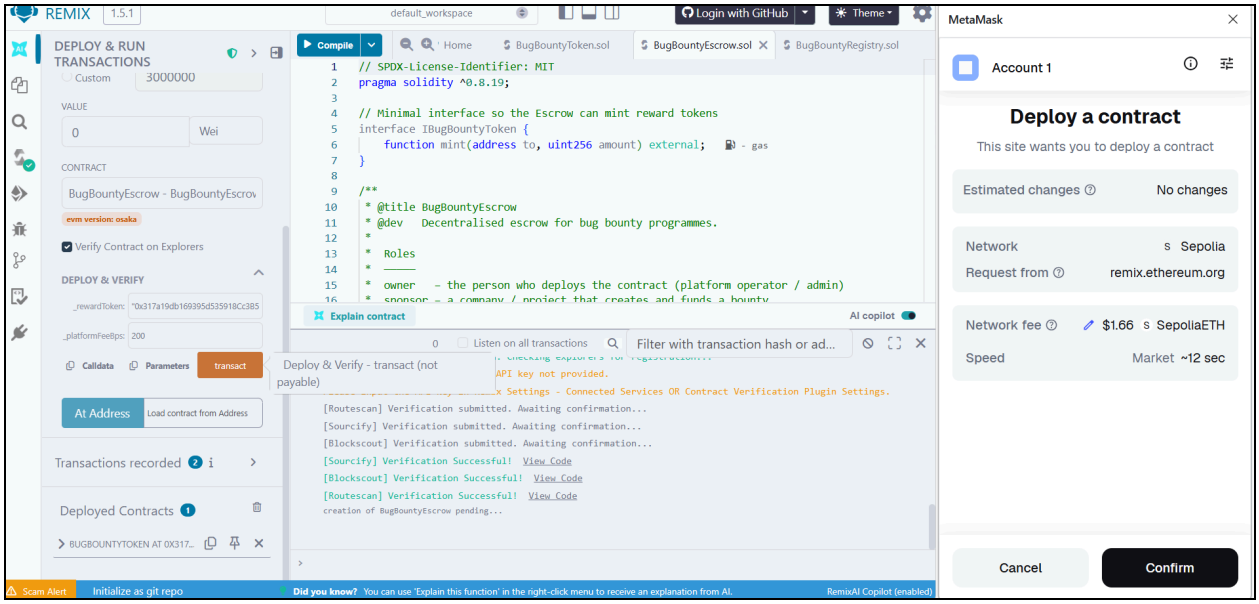

REMIX 1.5.1
default_workspace
Login with GitHub
Theme
DEPLOY & RUN TRANSACTIONS
Sepolia (11155111) network
ACCOUNT
0xc3d...6e81f (0.30210108 ETH)
+ Create Smart Account
GAS LIMIT
Estimated Gas
Custom
3000000
VALUE
0
Wei
CONTRACT
BugBountyToken - BugBountyToken.s
evm version: osaka
Verify Contract on Explorers
Deploy & Verify
At Address
Load contract from Address
Transactions recorded 1
Compile
Home
BugBountyToken.sol
BugBountyEscrow.sol
BugBountyRegistry.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;
/**
* @title BugBountyToken
* @dev Simple ERC20-style reward token for Bug Bounty hunters.
* Minted by the BugBountyEscrow contract when a bounty is paid out.
* Hunters accumulate BBT as reputation/reward tokens.
*/
contract BugBountyToken {
string public name = "Bug Bounty Token";
string public symbol = "BBT";
uint8 public decimals = 18;
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
You can use this terminal to:
• Check transactions details and start debugging.
• Execute JavaScript scripts:
- Input a script directly in the command line interface
- Select a Javascript file in the file explorer and then run `remix.execute()` or `remix.exeCurrent()` in the command line interface
- Right-click on a JavaScript file in the file explorer and then click `Run`
The following libraries are accessible:
• ethers.js
Type the library name to see available commands.
creation of BugBountyToken pending...
Scam Alert
Initialize as git repo
Did you know? You can use 'Explain this function' in the right-click menu to receive an explanation from AI.
RemixAI Copilot (enabled)
MetaMask
Account 1
Deploy a contract
This site wants you to deploy a contract
Estimated changes
No changes
Network
Sepolia
Request from
remix.ethereum.org
Network fee
$0.74 SepoliaETH
Speed
Market ~12 sec
Cancel
Confirm

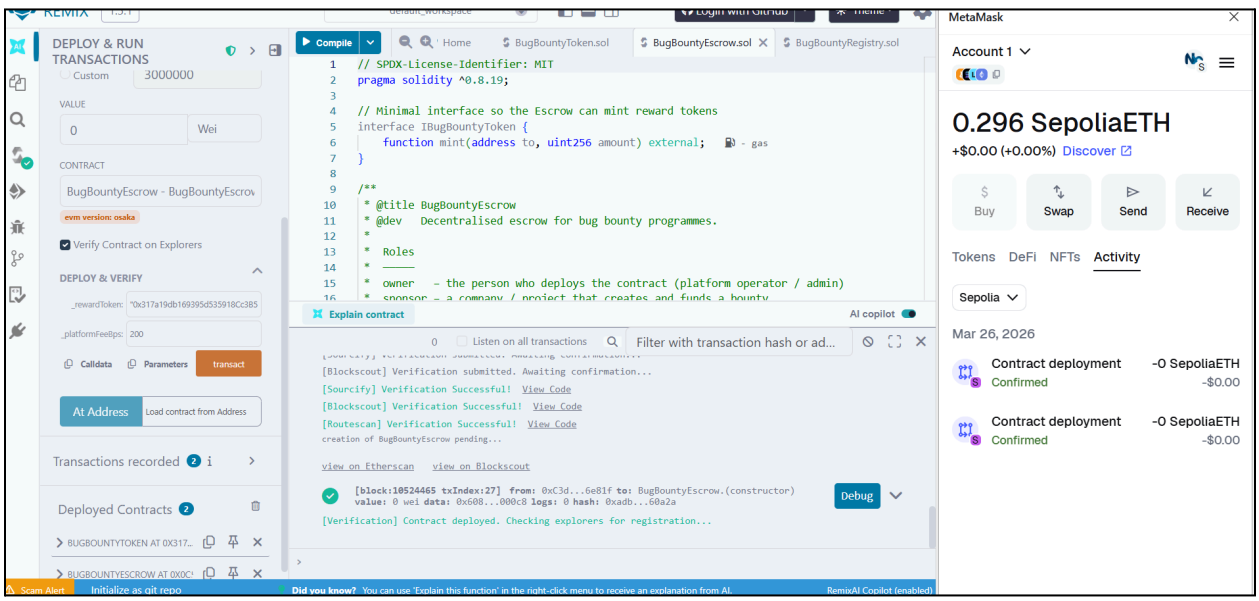


Step 3 — Deploy BugBountyEscrow

1. In the Contract dropdown, select **"BugBountyEscrow"**
2. You'll see two input fields before deploying:
   - _rewardToken → paste the **BugBountyToken address** from Step 1
   - _platformFeeBps → type **200** (means 2% fee)
3. Click **Deploy** → Confirm in MetaMask
4. **Copy and save its address**

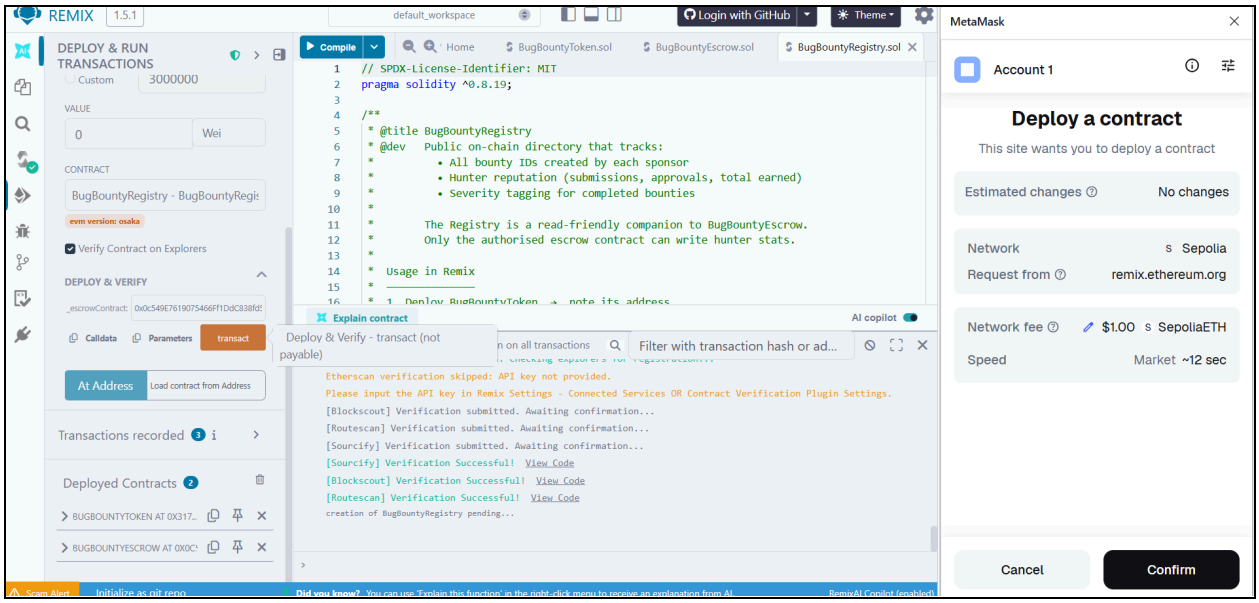


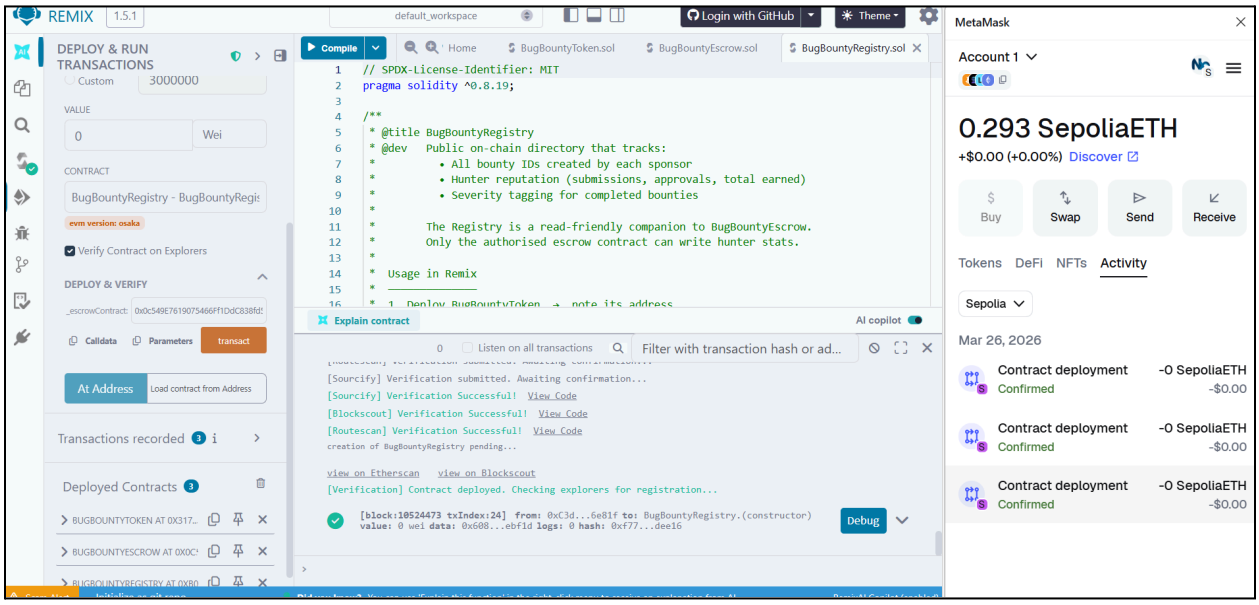




Step 4 — Deploy BugBountyRegistry

1. Select **"BugBountyRegistry"** in the dropdown
2. Input field:
   - _escrowContract → paste the **BugBountyEscrow address** from Step 2
3. Click **Deploy** → Confirm in MetaMask
4. **Copy and save its address**

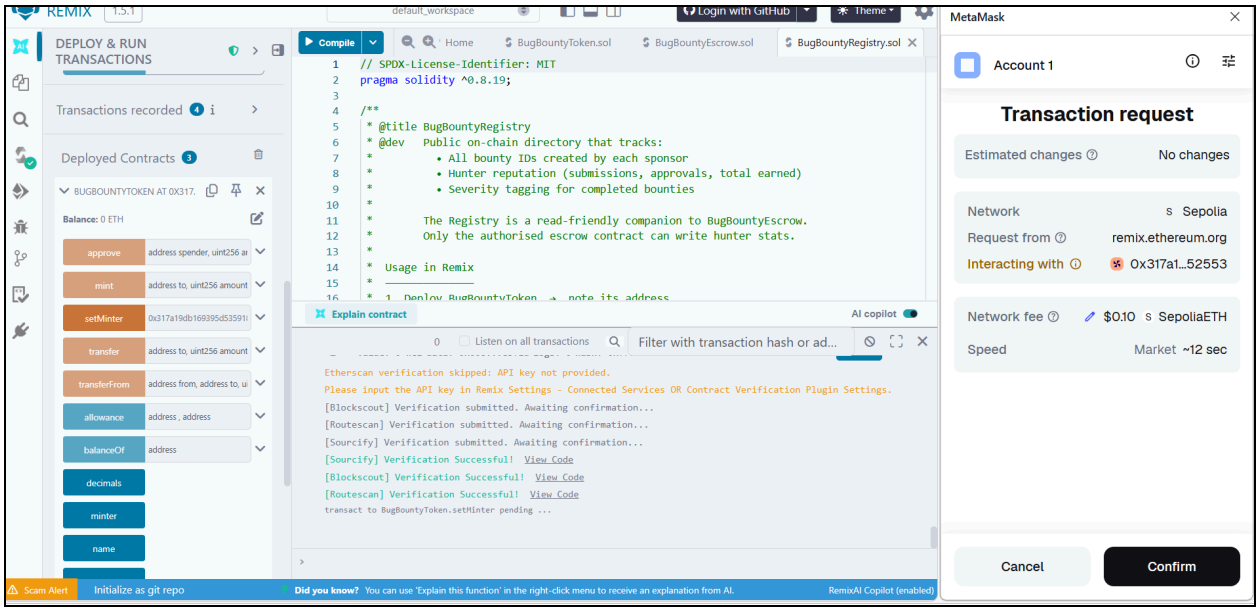


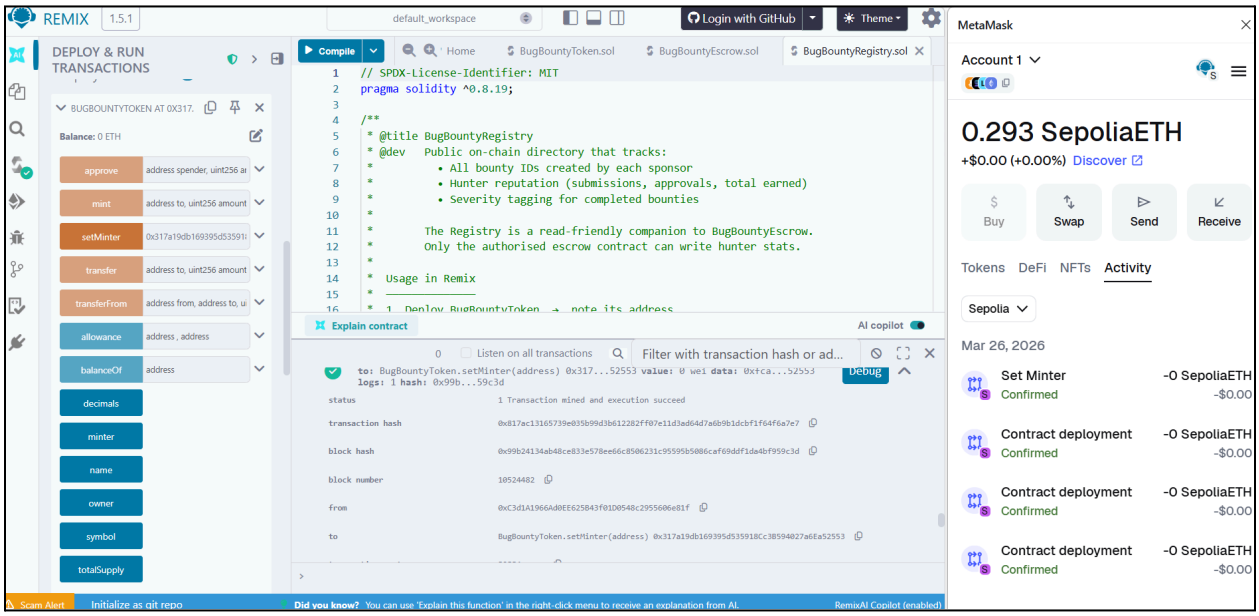


Step 5 — Link the Contracts Together

On BugBountyToken:

1. Expand it under Deployed Contracts
2. Find **setMinter** function
3. Paste the **BugBountyEscrow address** → click it → Confirm in MetaMask





Conclusion

The experiment successfully demonstrated MetaMask integration with Remix IDE and execution of blockchain transactions. It provided hands-on experience with smart contracts, ensuring secure and transparent interactions on a testnet.